

DBMS Lab Project Report

UCS310 – Database Management Systems

Smart Inventory Management for Retail Chain

by Group: 2C71

Tanish Kumar Singla (1024030759)

Ishaan Goel (1024030763)

Submitted to

Ms. Abhishelly Sharma



DEPARTMENT OF COMPUTER ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Deemed to be University)
Patiala, Punjab
India

Academic Year: 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives of the Project
4. Scope of the Project
5. Proposed System Description
6. Database Design
7. ER Diagram
8. Relational Schema
9. Normalization
10. SQL Implementation
11. Procedures
12. Functions
13. Triggers
14. Views and Reports
15. Transaction Management and Concurrency
16. Sample Queries
17. Expected Outcomes
18. Conclusion

1. Introduction

Inventory management plays an important role in retail businesses because it directly affects customer satisfaction, profitability, and operational efficiency. Retail chains often manage stock across multiple branches, making it difficult to maintain real-time visibility of products.

This project presents a Smart Inventory Management System for Retail Chains. The system centralizes inventory records across branches and automates operations such as stock tracking, sales processing, stock transfer, supplier management, reorder alerts, and expiry monitoring.

The database system ensures that stock levels remain accurate and up to date. It also provides analytical reports to help managers make better business decisions.

2. Problem Statement

Many retail chains maintain inventory data manually using spreadsheets or separate systems at different branches. This creates several issues:

- Lack of real-time stock visibility

- Difficulty in managing inventory across branches
- Frequent stockouts and overstocking
- Delayed reorder processes
- Poor supplier tracking
- Waste due to expired products
- Inconsistent and redundant data

3. Objectives of the Project

- To provide centralized inventory tracking across multiple branches
- To automatically generate reorder alerts when stock becomes low
- To maintain supplier records and ratings
- To support sales and stock transfer operations
- To track expiry dates of perishable goods
- To generate analytical reports for better decision making
- To maintain data consistency using normalization and constraints

4. Scope of the Project

The system covers:

- Product and category management
- Branch-wise inventory management
- Supplier management
- Sales processing
- Purchase order generation
- Stock transfer between branches
- Low stock alerts
- Expiry tracking
- Reporting and analytics

The system does not include HR, payroll, employee attendance, or delivery logistics.

5. Proposed System Description

The system uses a centralized database where every product is uniquely identified and linked with suppliers, categories, and inventory records.

Whenever a sale occurs, stock is automatically reduced. If the stock level falls below the reorder level, a trigger generates a purchase order automatically.

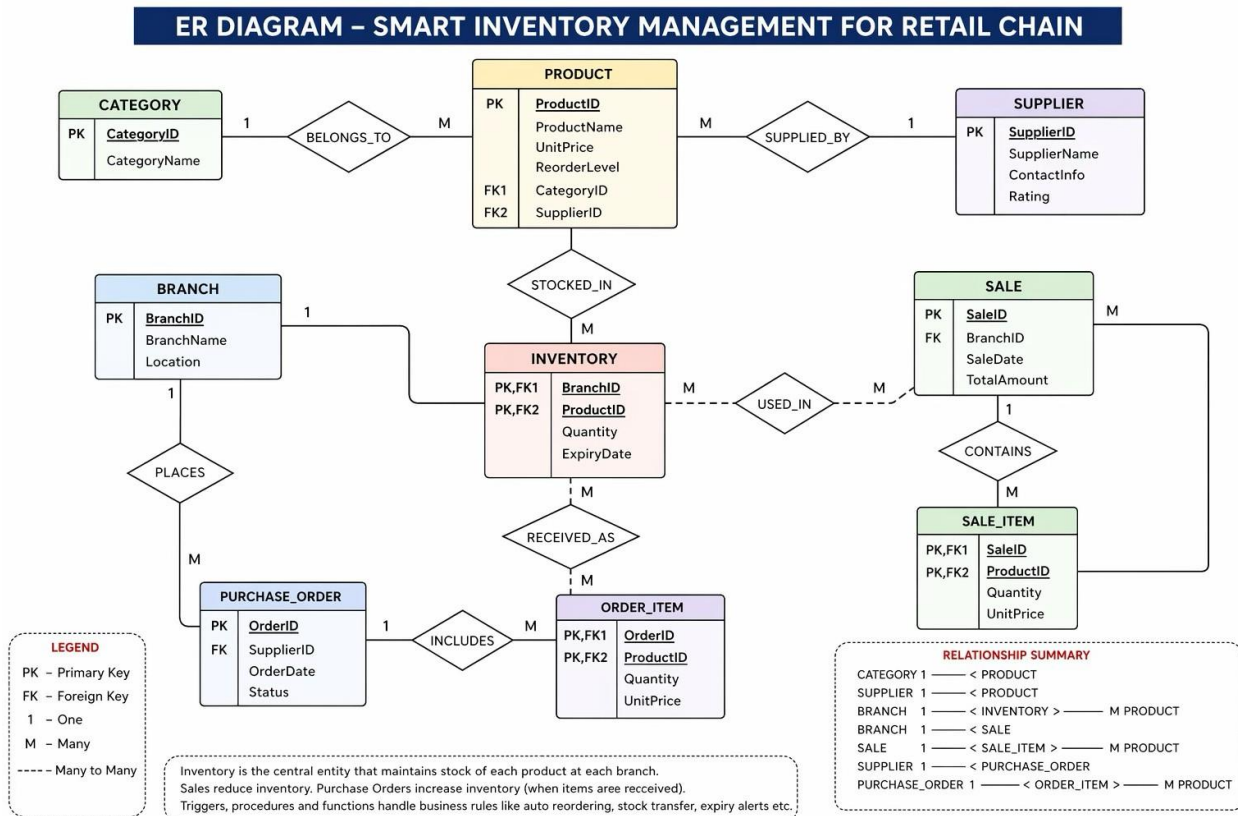
Managers can also transfer stock between branches and view reports related to sales, supplier performance, low stock products, and expiring products.

6. Database Design

The database contains the following main entities:

- Category
- Supplier
- Product
- Branch
- Inventory
- Sale
- Sale_Item
- Purchase_Order
- Order_Item
- Inventory_Log

7. ER Diagram



Main relationships:

- One category can have many products
- One supplier can supply many products
- One branch can contain many inventory records
- One product can appear in many inventory records
- One sale can contain many sale items
- One purchase order can contain many order items

8. Relational Schema

CATEGORY(CategoryID PK, CategoryName)

SUPPLIER(SupplierID PK, SupplierName, ContactInfo, Rating)

PRODUCT(ProductID PK, ProductName, UnitPrice, ReorderLevel, CategoryID FK, SupplierID FK)

BRANCH(BranchID PK, BranchName, Location)

INVENTORY(BranchID PK/FK, ProductID PK/FK, Quantity, ExpiryDate)

SALE(SaleID PK, BranchID FK, SaleDate, TotalAmount)

SALE_ITEM(SaleID PK/FK, ProductID PK/FK, Quantity, UnitPrice)

PURCHASE_ORDER(OrderID PK, SupplierID FK, OrderDate, Status)

ORDER_ITEM(OrderID PK/FK, ProductID PK/FK, Quantity, UnitPrice)

INVENTORY_LOG(LogID PK, BranchID, ProductID, OldQuantity, NewQuantity, ChangeDate, ActionType)

9. Normalization

The database is normalized up to Third Normal Form (3NF).

First Normal Form (1NF)

- All attributes are atomic
- No repeating groups are present

Second Normal Form (2NF)

- Partial dependency is removed
- Composite key tables like Inventory, Sale_Item, and Order_Item only contain attributes fully dependent on the complete key

Third Normal Form (3NF)

- No transitive dependency exists
- Supplier details are stored only in Supplier table
- Category details are stored only in Category table

Normalization reduces redundancy and improves data consistency.

10. SQL Implementation

The database uses SQL for:

- CREATE TABLE
- INSERT
- UPDATE
- DELETE
- ALTER
- SELECT
- JOIN
- VIEW
- INDEX

Constraints used:

- Primary Key
- Foreign Key
- Check Constraint
- Composite Key
- Default Constraint

11. Procedures

ProcessSale Procedure

This procedure:

- Checks stock availability
- Creates sale records
- Inserts into Sale_Item
- Updates inventory
- Uses transactions

TransferStock Procedure

This procedure:

- Transfers stock from one branch to another
- Checks stock availability
- Updates inventory at both branches
- Uses transactions

12. Functions

GetSupplierStatus Function

This function returns supplier status based on rating.

Example:

- Good
- Average

13. Triggers

AutoReorder Trigger

This trigger activates after inventory is updated.

If quantity falls below reorder level:

- A purchase order is generated
- An order item is added automatically

CheckStockBeforeSale Trigger

This trigger checks stock before a sale item is inserted.

If stock is insufficient:

- The sale is rejected

LogInventoryChanges Trigger

This trigger records all stock changes into the Inventory_Log table.

14. Views and Reports

LowStockProducts View

Displays products where quantity is less than reorder level.

ExpiringProducts View

Displays products nearing expiry.

Reports Generated

- Branch-wise sales report
- Monthly sales trend report
- Top selling products report
- Supplier performance report
- Low stock report
- Near expiry report

15. Transaction Management and Concurrency

The system follows ACID properties:

- Atomicity: All operations succeed or fail together
- Consistency: Constraints ensure valid data
- Isolation: Multiple transactions do not interfere with each other
- Durability: Committed data remains saved permanently

Transactions are used in sales and stock transfer procedures.

16. Sample Queries

Top Selling Products

```
SELECT p.ProductName, SUM(si.Quantity)
FROM Sale_Item si
JOIN Product p ON si.ProductID = p.ProductID
GROUP BY p.ProductName;
```

Low Stock Products

```
SELECT *
FROM LowStockProducts;
```

Expiring Products

```
SELECT *  
FROM ExpiringProducts;
```

Branch Wise Sales

```
SELECT b.BranchName, SUM(s.TotalAmount)  
FROM Sale s  
JOIN Branch b ON s.BranchID = b.BranchID  
GROUP BY b.BranchName;
```

17. Expected Outcomes

- Better stock visibility
- Reduced stockouts and overstocking
- Faster reorder process
- Reduced wastage of perishable items
- Better supplier tracking
- Better reporting and business decisions
- Improved operational efficiency

18. Conclusion

The Smart Inventory Management System for Retail Chain is a complete database solution for managing inventory, suppliers, branches, sales, and purchase orders.

The system uses SQL, procedures, triggers, functions, and views to automate operations and maintain data integrity.

By centralizing inventory records and providing automated alerts and reports, the project improves efficiency, reduces manual work, and supports better decision making.

This project demonstrates strong use of database concepts such as normalization, constraints, transactions, triggers, procedures, and relational design.